Reviewer Pack — Schur-Weyl Multiplicity Gap in Symmetric Powers of Permutati...

Entry b0bc1c3eee2c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 10:01:15 UTC

Schur-Weyl Multiplicity Gap in Symmetric Powers of Permutation Matrices

Entry ID: `b0bc1c3eee2c` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 10:01:15 UTC

1. Conjecture

Field A (mathematical branch): Schur-Weyl Duality **Field B** (complexity object): Monotone Circuit Complexity

Statement:

> For n ≥ 2, the multiplicity of the trivial representation in the decomposition of Sym^k(Perm_n) exceeds that in Sym^k(Det_n) by Ω(n^{1/2}) for all k ≤ n^{1/2}, where Perm_n and Det_n are the n×n permutation and determinant matrices. This gap persists under linear substitutions preserving the symmetric group action.

Rationale (proposer's reasoning):

> Schur-Weyl duality decomposes tensor powers into irreducible representations, revealing structural asymmetry between determinant (alternating) and permanent (symmetric) under symmetric group actions. The multiplicity gap reflects differing symmetry constraints, potentially exposing algebraic barriers to monotone circuit simulation.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ccecb7de0204ff9e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM_1	LLM_2
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def is_permutation(matrix):
        n = len(matrix)
        identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
        product = matrix_multiply(matrix, transpose(matrix))
        return all(abs(product[i][j] - identity[i][j]) < 1e-9 for i in range(n) for j in range(n))

    def transpose(matrix):
        n = len(matrix)
        return [[matrix[j][i] for j in range(n)] for i in range(n)]

    def matrix_multiply(A, B):
        n = len(A)
        C = [[0 for _ in range(n)] for _ in range(n)]
        for i in range(n):
            for j in range(n):
                for k in range(n):
                    C[i][j] += A[i][k] * B[k][j]
        return C

    def determinant(matrix):
        n = len(matrix)
        if n == 1:
            return matrix[0][0]
        det = 0
        for j in range(n):
            minor = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += (-1) ** j * matrix[0][j] * determinant(minor)
        return det

    def symmetric_power(matrix, k):
        result = matrix
```

```
    for _ in range(k-1):
        result = matrix_multiply(result, matrix)
    return result

def multiplicity_of_trivial_representation(matrix):
    n = len(matrix)
    identity = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
    symmetric_power_matrix = symmetric_power(matrix, n // 2)
    eigenvalues = []
    A = matrix_multiply(symmetric_power_matrix, transpose(identity))
    Q, R = gram_schmidt(A)
    for v in Q:
        if norm(v) > 1e-9:
            eigenvalues.append(Fraction(1, len(Q)))
    return sum(eigenvalues)

def gram_schmidt(A):
    n = len(A)
    Q = []
    R = [[0 for _ in range(n)] for _ in range(n)]
    for i in range(n):
        v = A[i]
        for j in range(i):
            r_ij = sum(Q[j][k] * v[k] for k in range(n))
            R[j][i] = r_ij
            v = [v[k] - r_ij * Q[j][k] for k in range(n)]
        norm_v = norm(v)
        if norm_v > 1e-9:
            Q.append([x / norm_v for x in v])
            R[i][i] = norm_v
    return Q, R

def norm(vector):
    return math.sqrt(sum(x**2 for x in vector))

n = random.randint(5, 40)
perm_matrix = [[1 if i == j else 0 for j in range(n)] for i in range(n)]
det_matrix = [[1 if i == j else 0 for j in range(n)] for i in range(n)]

if not is_permutation(perm_matrix) or determinant(det_matrix) != 1:
    return {
        "metric_name": "Multiplicity Gap",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

perm_multiplicity = multiplicity_of_trivial_representation(perm_matrix)
det_multiplicity = multiplicity_of_trivial_representation(det_matrix)
gap = perm_multiplicity - det_multiplicity

return {
    "metric_name": "Multiplicity Gap",
    "metric_value": gap,
    "instances_tested": 1,
```

```
        "conjecture_holds": gap > math.sqrt(n),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for r in results:
            if not r["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"Multiplicity gap does not exceed Ω(n^{1
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test timed out before producing results, preventing evaluation of support fraction or counterexamples. | next: Optimize test parameters or increase time limit to resolve timeout

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	53579
2	preregistration	ollama_remote	qwen3:8b	0	0	27549
3	novelty	ollama_remote	qwen3:8b	0	0	24173
4	novelty	ollama_remote	qwen3:8b	0	0	22435
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10895
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9822
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15664
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13962
9	judge	ollama_remote	qwen3:8b	0	0	19911

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 197990 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b0bc1c3eee2c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b0bc1c3eee2c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/b0bc1c3eee2c.tar.gz` (if generated)